RDF 1.2 Turtle

Terse RDF Triple Language

W3C Working Draft 29 January 2026

▼ **More details about this document**

This version:
https://www.w3.org/TR/2026/WD-rdf12-turtle-20260129/

Latest published version:
https://www.w3.org/TR/rdf12-turtle/

Latest editor's draft:
https://w3c.github.io/rdf-turtle/spec/

History:
https://www.w3.org/standards/history/rdf12-turtle/
Commit history

Test suite:
https://w3c.github.io/rdf-tests/rdf/rdf12/rdf-turtle/

Latest Recommendation:
https://www.w3.org/TR/turtle

Editors:
Gregg Kellogg (until 2025-09-06), in memoriam
Dominik Tomaszuk

Former editors:
Eric Prud'hommeaux (RDF 1.1)
Gavin Carothers (RDF 1.1)

Authors:
David Beckett
Tim Berners-Lee (W3C)
Eric Prud'hommeaux (W3C)
Gavin Carothers (Lex Machina, Inc)

Feedback:
GitHub w3c/rdf-turtle (pull requests, new issue, open issues)
public-rdf-star-wg@w3.org with subject line `[rdf12-turtle] … message topic …` (archives)

Copyright © 2008-2026 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Abstract

The Resource Description Framework (RDF) is a general-purpose language for representing information in the Web.

This document defines a textual syntax for RDF called Turtle that allows an RDF graph to be completely written in a compact and natural text form, with abbreviations for common usage patterns and datatypes. Turtle provides levels of compatibility with the N-Triples [*RDF12-N-TRIPLES*] format as well as the triple pattern syntax of *SPARQL 1.2 Query Language* W3C Recommendation.

RDF 1.2 Turtle introduces triple terms as a fourth kind of RDF term which can be used as the object of another triple, making it possible to make statements about other statements. Triple terms are typically not used explicitly as the `reifiedTriple` construction is generally preferred. RDF 1.2 Turtle also adds support for directional language-tagged strings.

In addition, RDF 1.2 Turtle introduces an annotation syntax which allows triple terms to also be asserted.

Status of This Document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the W3C standards and drafts index.

This document is a part of the RDF 1.2 document suite. The document defines Turtle, the Terse RDF Triple Language, a concrete syntax for RDF [*RDF12-CONCEPTS*].

This document was published by the RDF & SPARQL Working Group as a Working Draft using the Recommendation track.

Publication as a Working Draft does not imply endorsement by W3C and its Members.

This is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to cite this document as other than a work in progress. Future updates to this upcoming Recommendation may incorporate new features.

This document was produced by a group operating under the W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 18 August 2025 W3C Process Document.

§ Set of Documents

This document is one of eleven RDF 1.2 and twelve SPARQL 1.2 documents produced by the RDF & SPARQL Working Group.

▶ **List of documents**

Table of Contents

Abstract

Status of This Document
Set of Documents

1. Introduction

2. Turtle Language
2.1 Simple Triples
2.2 Predicate Lists
2.3 Object Lists
2.4 Version Announcement
2.5 IRIs
2.6 RDF Literals
2.6.1 Quoted Literals
2.6.2 Numbers
2.6.3 Booleans
2.7 RDF Blank Nodes
2.8 Nesting of Blank Nodes without Blank Node Identifiers
2.9 Collections
2.10 Triple Terms
2.11 Reifying Triples
2.11.1 Annotation Syntax

3. **Examples**

4. **Turtle compared to SPARQL**

5. **Conformance**
5.1 Media Type and Content Encoding

6. **Turtle Grammar**
6.1 White Space
6.2 Comments
6.3 IRI References
6.4 Escape Sequences
6.5 Grammar
6.6 Selected Terminal Literal Strings

7. **Parsing**
7.1 Parser State
7.2 RDF Term Constructors
7.3 RDF Triples Constructors
7.3.1 Reifiers
7.3.2 Reified Triples
7.3.3 Annotations
7.3.4 Annotation Blocks
7.3.5 Property Lists
7.3.6 Collections
7.4 Parsing Example

A. **Embedding Turtle in HTML documents**
A.1 XHTML
A.2 Parsing Turtle in HTML

B. **Privacy Considerations**

C. **Security Considerations**

D. **Internet Media Type, File Extension and Macintosh File Type**

E. **Acknowledgments**
E.1 Acknowledgments for RDF 1.1
E.2 Acknowledgments for RDF 1.2

F. **Changes between RDF 1.1 and RDF 1.2**

G. **Index**
G.1 Terms defined by this specification
G.2 Terms defined by reference

H. **References**
H.1 Normative references
H.2 Informative references

§ 1. Introduction

This section is non-normative.

This document defines Turtle, the Terse RDF Triple Language, a concrete syntax for RDF [*RDF12-CONCEPTS*].

A Turtle document is a textual representations of an RDF graph. The following Turtle document describes the relationship between Green Goblin and Spiderman.

EXAMPLE 1: Example Turtle

```
BASE <http://example.org/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rel: <http://www.perceive.net/schemas/relationship/>

<#green-goblin>
  rel:enemyOf <#spiderman> ;
  a foaf:Person ;    # in the context of the Marvel universe
  foaf:name "Green Goblin" .

<#spiderman>
  rel:enemyOf <#green-goblin> ;
  a foaf:Person ;
  foaf:name "Spiderman", "Человек-паук"@ru .
```

This example introduces many of features of the Turtle language: BASE and Relative IRI references, PREFIX and prefixed names, predicate lists separated by `;`, object lists separated by `,`, the token `a`, and literals.

The Turtle grammar for `triples` is a subset of *SPARQL 1.2 Query Language* [*SPARQL12-QUERY*] grammar for `TriplesBlock`. The two grammars share production and terminal names where possible.

The construction of an RDF graph from a Turtle document is defined in Turtle Grammar and Parsing.

§ 2. Turtle Language

This section is non-normative.

A Turtle document allows writing down an RDF graph in a compact textual form. An RDF graph is made up of triples consisting of a subject, predicate and object.

Comments may follow a `#` that is not part of another lexical token, and continue to the end of the line.

§ 2.1 Simple Triples

The simplest triple statement is a sequence of (subject, predicate, and object) terms, forming an RDF triple, terminated by a period (`.`). White space (`spaces`, `tabs`, `LFs`, and/or `CRs`) may surround terms, except where significant as noted in the grammar.

EXAMPLE 2: Simple Triple

```
<http://example.org/#spiderman>
  <http://www.perceive.net/schemas/relationship/enemyOf>
    <http://example.org/#green-goblin> .
```

§ 2.2 Predicate Lists

Often the same subject will be referenced by a number of predicates. The `predicateObjectList` production matches a series of predicates and objects, separated by `;`, following a subject. This expresses a series of RDF Triples with that subject and each predicate and object allocated to one triple. Thus, the `;` is used to repeat the subject of triples that vary only in predicate and object RDF terms.

These two examples are equivalent ways of writing the triples about Spiderman.

EXAMPLE 3: Predicate List

```
<http://example.org/#spiderman> <http://www.perceive.net/schemas/relationship/enemyOf> <h
  <http://xmlns.com/foaf/0.1/name> "Spiderman" .
```

EXAMPLE 4: Predicate List expanded to Simple Triples

```
<http://example.org/#spiderman> <http://www.perceive.net/schemas/relationship/enemyOf> <h
<http://example.org/#spiderman> <http://xmlns.com/foaf/0.1/name> "Spiderman" .
```

§ 2.3 Object Lists

As with predicates, the same subject and predicate often repeat with different objects. The `objectList` production matches a series of objects separated by `,` following a predicate. This expresses a series of RDF Triples with the same subject and predicate and each object allocated to one triple. Thus, the `,` is used to repeat the subject and predicate of triples that only differ in the object RDF term.

These two examples are equivalent ways of writing Spiderman's name in two languages.

EXAMPLE 5: Object List

```
<http://example.org/#spiderman> <http://xmlns.com/foaf/0.1/name> "Spiderman", "Человек-па
```

EXAMPLE 6: Object List expanded to Simple Triples

```
<http://example.org/#spiderman> <http://xmlns.com/foaf/0.1/name> "Spiderman" .
<http://example.org/#spiderman> <http://xmlns.com/foaf/0.1/name> "Человек-паук"@ru .
```

There are four RDF terms defined in RDF Concepts: IRIs (Internationalized Resource Identifiers), literals, blank nodes, and triple terms; Turtle provides a number of ways of writing each.

§ 2.4 Version Announcement

The Turtle language has evolved since its origin, and RDF 1.2 adds new syntax. RDF 1.2 Turtle introduces the `VERSION` and `@version` directives along with an optional `version` Media Type parameter. When serializing Turtle with new features such as initial text direction or triple terms, authors can announce the use of the new syntax forms using these directives.

EXAMPLE 7: Version announcement

```
VERSION "1.2"
PREFIX : <http://example.com/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
:a :name "Alice" ~ :t {| :statedBy :bob ; :recorded "2021-07-07"^^xsd:date |} .
```

Alternatively, the old-style directive can be used (note that there is a trailing "`.`"):

EXAMPLE 8: Old-style version announcement

```
@version "1.2" .
@prefix : <http://example.com/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
:a :name "Alice" ~ :t {| :statedBy :bob ; :recorded "2021-07-07"^^xsd:date |} .
```

When providing content over HTTP, servers can announce the version using the optional `version` Media Type parameter:

EXAMPLE 9: HTTP version announcement

```
GET /document.ttl HTTP/1.1
Host: example.com
Accept: text/turtle; version=1.2
```

When using RDF 1.2-specific features, such as initial text direction, the specific RDF version should be announced using either `VERSION` or `@version` directives early in the document. This allows parsers that do not support these features to detect the presense of such features early, and potentially inform the user, giving them an opportunity to stop the job or otherwise act on the fact that some amount of the input data will not be processed as desired.

Multiple version directives may appear in a Turtle document. The directive applies to the part of the document following the directive, until another directive is encountered or the end of the document is reached.

In the absence of a current version directive, any version specified as part of the Media Type is considered. In the absence of any version information, a sensible default is `"1.1"`, to maximize compatibility with the previous version of Turtle [*TURTLE*]. In any case, the version announcement is merely a hint; parsers are not required to reject features that are outside the announced version (but could signal them with a warning).

NOTE

Version labels are defined in *RDF 1.2 Concepts and Abstract Data Model* section 2.1 Version Labels. Processors may treat unrecognized labels as an error or as a warning.

While `"1.1"` is an acceptable version label, its use in a `VERSION` or `@version` directive is discouraged, as it would needlessly cause Turtle 1.1 parsers to fail.

§ 2.5 IRIs

IRIs may be written as resolved IRIs, Relative IRI references, or prefixed names. Relative and resolved IRIs are preceded by <, followed by >, and may contain numeric escape sequences (described below). For example, `<http://example.org/#green-goblin>`.

Relative IRI references like `<#green-goblin>` are resolved relative to the current base IRI. A new base IRI can be defined using the `@base` or `BASE` directive. Specifics of this operation are defined in 6.3 IRI References.

The token `a` in the predicate position of a Turtle triple represents the IRI `http://www.w3.org/1999/02/22-rdf-syntax-ns#type` .

A ***prefixed name*** is a prefix label and a local part, separated by a `:`. A prefixed name is turned into an IRI by concatenating the IRI associated with the prefix and the local part. The `@prefix` or `PREFIX` directive associates a prefix label with an IRI. Subsequent `@prefix` or `PREFIX` directives may re-map the same prefix label.

> NOTE
>
> The Turtle language originally permitted only the syntax including the `@` character for writing prefix and base directives. The case-insensitive `PREFIX`, `BASE`, and `VERSION` forms were added to align Turtle's syntax with that of SPARQL. The original directives `@prefix`, `@base`, or `@version` may also be used. Using `PREFIX`, `BASE`, and `VERSION` may make it easier to copy declarations to SPARQL queries.

To write `http://www.perceive.net/schemas/relationship/enemyOf` using a prefixed name:

1. Define a prefix label for the vocabulary IRI `http://www.perceive.net/schemas/relationship/` as `somePrefix`
2. Then write `somePrefix:enemyOf` which is equivalent to writing `<http://www.perceive.net/schemas/relationship/enemyOf>`

This can be written using either the SPARQL-style syntax for prefix declarations:

EXAMPLE 10: SPARQL syntax for prefix declarations

```
PREFIX somePrefix: <http://www.perceive.net/schemas/relationship/>

<http://example.org/#green-goblin> somePrefix:enemyOf <http://example.org/#spiderman> .
```

or original Turtle syntax for prefix declarations:

EXAMPLE 11: Original syntax for prefix declarations

```
@prefix somePrefix: <http://www.perceive.net/schemas/relationship/> .

<http://example.org/#green-goblin> somePrefix:enemyOf <http://example.org/#spiderman> .
```

> NOTE
>
> Prefixed names are a superset of XML QNames. They differ in that the local part of prefixed names may include:
>
> - leading digits, e.g., `leg:3032571` or `isbn13:9780136019701`
> - non leading colons, e.g., `og:video:height`
> - reserved character escape sequences, e.g., `wgs:lat\-long`

The following Turtle document contains examples of all the different ways of writing IRIs in Turtle.

EXAMPLE 12: Different ways of writing IRIs

```
# A triple with all resolved IRIs
<http://one.example/subject1> <http://one.example/predicate1> <http://one.example/object1

@base <http://one.example/> .
<subject2> <predicate2> <object2> .     # relative IRI references, e.g., http://one.examp

BASE <http://one.example/>
<subject2> <predicate2> <object2> .     # relative IRI references, e.g., http://one.examp

@prefix p: <http://two.example/> .
p:subject3 p:predicate3 p:object3 .     # prefixed name, e.g., http://two.example/subject

PREFIX p: <http://two.example/>
p:subject3 p:predicate3 p:object3 .     # prefixed name, e.g., http://two.example/subject

@prefix p: <path/> .                    # prefix p: now stands for http://one.example/pat
p:subject4 p:predicate4 p:object4 .     # prefixed name, e.g., http://one.example/path/su

PrEfIx : <http://another.example/>       # empty prefix
:subject5 :predicate5 :object5 .        # prefixed name, e.g., http://another.example/sub

:subject6 a :subject7 .                 # same as :subject6 <http://www.w3.org/1999/02/22

<http://伝言.example/?user=¡&channel=R%26D> a :subject8 . # a multi-script subject IRI .
```

NOTE

While the `@prefix` and `@base` directives require a trailing `.` after the IRI, the equivalent `PREFIX` and `BASE` do not have a trailing `.` after the IRI part of the directive. The `PREFIX` and `BASE` are case-insensitive and can be written as `prefix` and `base` or as any blend of mixed case.

§ 2.6 RDF Literals

Literals are used to identify values such as strings, numbers, dates.

EXAMPLE 13: Literals in Turtle

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

<http://example.org/#green-goblin> foaf:name "Green Goblin" .

<http://example.org/#spiderman> foaf:name "Spiderman" .
```

§ 2.6.1 Quoted Literals

Quoted Literals (Grammar production `RDFLiteral`) have a lexical form followed by a language tag (possibly including an initial text direction), a datatype IRI, or neither.

The representation of the lexical form consists of an initial delimiter (e.g., ", ', """, or '''); a sequence of permitted characters, numeric escape sequences, and/or string escape sequences; and a final delimiter matching the initial delimiter. The corresponding RDF lexical form is the characters between the delimiters, after processing any escape sequences.

If present, the language tag is preceded by an @ and may be followed by an initial text direction separated from the language tag by --.

If there is no language tag, there may be a datatype IRI, preceded by ^^. A datatype IRI in Turtle may be written using either a resolved IRI, a relative IRI reference, or a prefixed name.

If there is no datatype IRI and no language tag, the datatype is `xsd:string`.

A \ may not appear in any quoted literal except as part of an escape sequence. Other restrictions depend on the delimiter:

- Literals delimited by ' may not contain unescaped ', LF, nor CR characters.
- Literals delimited by " may not contain unescaped ", LF, nor CR characters.
- Literals delimited by ''' may not contain such a sequence.
- Literals delimited by """ may not contain such a sequence.

EXAMPLE 14: Quoted Literals

```
VERSION "1.2"
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX show: <http://example.org/vocab/show/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

show:218 rdfs:label "That Seventies Show"^^xsd:string .               # literal with XML
show:218 rdfs:label "That Seventies Show"^^<http://www.w3.org/2001/XMLSchema#string> . #
show:218 rdfs:label "That Seventies Show" .                                            #
show:218 show:localName "That Seventies Show"@en .                    # literal with a l
show:218 show:localName "HTML היא שפת סימון."@he--rtl .               # literal with a l
show:218 show:localName "That Seventies Show"@en-us--ltr .            # literal with a l
show:218 show:localName 'Cette Série des Années Soixante-dix'@fr .    # literal delimite
show:218 show:localName "Cette Série des Années Septante"@fr-be .     # literal with a r
show:218 show:blurb '''This is a multi-line
                       literal with many quotes (""""")
                       and up to two sequential apostrophes ('').''' . # literal with emb
```

§ 2.6.2 Numbers

Numbers can be written like other literals with lexical form and datatype (e.g., `"-5.0"^^xsd:decimal`). Turtle has a shorthand syntax for writing integer values, arbitrary precision decimal values, and double precision floating point values.

Data Type	Abbreviated	Lexical	Description
xsd:integer	`-5`	`"-5"^^xsd:integer`	Integer values may be written as an optional sign and a series of digits. Integers match the regular expression "`[+-]?[0-9]+`".
xsd:decimal	`-5.0`	`"-5.0"^^xsd:decimal`	Arbitrary-precision decimals may be written as an optional sign, zero or more digits, a decimal point, and one or more digits. Decimals match the regular expression "`[+-]?[0-9]*\.[0-9]+`".

xsd:double	`4.2E9`	`"4.2E9"^^xsd:double`	Double-precision floating point values may be written as an optionally signed mantissa with an optional decimal point, the letter `e` or the letter `E`, and an optionally signed integer exponent. The exponent matches the regular expression "`[+-]?[0-9]+`" and the mantissa matches one of these regular expressions: "`[+-]?[0-9]+\.[0-9]+`", "`[+-]?\.[0-9]+`", or "`[+-]?[0-9]`".

EXAMPLE 15: Number Literals

```
PREFIX : <http://example.org/elements/>
<http://en.wikipedia.org/wiki/Helium>
    :atomicNumber 2 ;               # xsd:integer
    :atomicMass 4.002602 ;          # xsd:decimal
    :specificGravity 1.663E-4 .     # xsd:double
```

2.6.3 Booleans

Boolean values may be written as either `true` or `false` (case-sensitive) and represent RDF literals with the datatype xsd:boolean [*XMLSCHEMA11-2*].

EXAMPLE 16: Boolean Literals

```
PREFIX : <http://example.org/stats/>
<http://somecountry.example/census2007>
    :isLandlocked false .           # xsd:boolean
```

2.7 RDF Blank Nodes

RDF blank nodes in Turtle are expressed as `_:` followed by a blank node identifier which is a series of characters. The characters in the identifier are built upon `PN_CHARS_BASE`, liberalized as follows:

- The character `_` and the digit characters `0`–`9` may appear anywhere in a blank node identifier.
- The character `.` may appear anywhere except the first or last character.
- The characters ‿, ·, ⁀, ⁔, and the combining diacritical marks (`U+0300` to `U+036F`) are permitted anywhere except the first character.

A fresh RDF blank node is allocated for each unique blank node identifier in a document. Repeated use of the same blank node identifier identifies the same blank node.

EXAMPLE 17: Blank Node in Turtle

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

_:alice foaf:knows _:bob .
_:bob foaf:knows _:alice .
```

§ 2.8 Nesting of Blank Nodes without Blank Node Identifiers

In Turtle, fresh RDF blank nodes are also allocated when matching the production `blankNodePropertyList` and the terminal `ANON`. Both of these may appear in the `subject` or `object` position of a triple (see the Turtle Grammar). That subject or object is a fresh RDF blank node. This blank node also serves as the subject of the triples produced by matching the `predicateObjectList` production embedded in a `blankNodePropertyList`. The generation of these triples is described in Predicate Lists. Blank nodes are also allocated for collections described below.

EXAMPLE 18: Blank Node Property List

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

# Someone knows someone else, who has the name "Bob".
[] foaf:knows [ foaf:name "Bob" ] .
```

The Turtle grammar allows `blankNodePropertyList`s to be nested. In this case, each inner `[` establishes a new subject blank node which reverts to the outer node at the `]`, and serves as the current subject for predicate object lists.

The use of `predicateObjectList` within a `blankNodePropertyList` is a common idiom for representing a series of properties of a node.

Abbreviated:

EXAMPLE 19

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

[ foaf:name "Alice" ] foaf:knows [
    foaf:name "Bob" ;
    foaf:knows [
        foaf:name "Eve" ] ;
    foaf:mbox <mailto:bob@example.com> ] .
```

Corresponding simple triples:

EXAMPLE 20

```
_:a <http://xmlns.com/foaf/0.1/name> "Alice" .
_:a <http://xmlns.com/foaf/0.1/knows> _:b .
_:b <http://xmlns.com/foaf/0.1/name> "Bob" .
_:b <http://xmlns.com/foaf/0.1/knows> _:c .
_:c <http://xmlns.com/foaf/0.1/name> "Eve" .
_:b <http://xmlns.com/foaf/0.1/mbox> <mailto:bob@exampl
```

§ 2.9 Collections

RDF provides a Collection [*RDF12-SEMANTICS*] structure for lists of RDF nodes. The Turtle syntax for Collections is a possibly empty list of RDF terms enclosed by `()`. This collection represents an `rdf:first`/`rdf:rest` list structure with the sequence of objects of the `rdf:first` statements being the order of the terms enclosed by `()`.

The `(…)` syntax *MUST* appear in the `subject` or `object` position of a triple (see the Turtle Grammar). The blank node at the head of the list is the subject or object of the containing triple.

EXAMPLE 21: Collections in Turtle

```
PREFIX : <http://example.org/foo/>
# the object of this triple is the RDF collection blank node
:subject :predicate ( :a :b :c ) .

# an empty collection value - rdf:nil
:subject :predicate2 () .
```

§ 2.10 Triple Terms

A triple term may be the object of an RDF triple.

A triple term is represented as a `tripleTerm` with `ttSubject`, `predicate`, and `ttObject`, all preceded by `<<(`, and all followed by `)>>`. Note that triple terms may be nested.

EXAMPLE 22: Triple Term

```
VERSION     "1.2"
PREFIX :    <http://www.example.org/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

_:e38  :familyName                    "Smith" .
_:anno rdf:reifies <<( _:e38 :jobTitle "Designer" )>> .
_:anno :accordingTo                    _:e22 .
```

§ 2.11 Reifying Triples

In RDF, triple terms are rarely used directly, as they are generally restricted to be used only as the object of a triple using the `rdf:reifies` predicate. Such a triple is called a ***reifying triple***. Turtle provides a shorthand notation for writing reifying triples using the `reifiedTriple` production.

A `reifiedTriple` is syntactic sugar representing a reifying triple, which defines a specific relationship between an identifier (***reifier***) and a triple term. The identifier becomes a way to indirectly refer to a triple term, which may or may not be asserted within the graph corresponding to this input document.

NOTE

Reification in RDF 1.2 is a concept distinct from the Reification vocabulary originally defined in RDF Semantics. While both terms describe a representation of an RDF triple using components, RDF 1.2 uses the term to identify a triple term using the `rdf:reifies` predicate.

A reifying triple is represented using the `reifiedTriple` production starting with `<<`, followed by a `rtSubject`, a `predicate`, and a `rtObject`, followed by an optional `reifier`, composed of a `~` followed by an optional `iri` production or `BlankNode` production, and ending with `>>`. For example, `<< :subject :predicate :object ~ :IRIREF >>`. If no reifiers are present, or a `reifier` is not immediately followed by an `iri` or `BlankNode`, a fresh RDF blank node is allocated, as with `<< :subject :predicate :object >>`, or `<< :subject :predicate :object ~ >>`.

NOTE

reifiedTriples may be nested, like
<< :subject1 :predicate1 << :subject2 :predicate2 :object2 >> ~:IRIREF1 >> or
<< :subject4 :predicate4 << :subject3 :predicate3 :object3 ~:IRIREF3 >> >>.

If a reifiedTriple is not identified by an IRI or blank node, a fresh RDF blank node is allocated and used to identify this relationship.

EXAMPLE 23: Reified Triple

```
VERSION     "1.2"
PREFIX :    <http://www.example.org/>

:employee38 :familyName "Smith" .
<< :employee38 :jobTitle "Assistant Designer" >> :accordingTo :employee22 .
```

EXAMPLE 24: Reified Triple with explicit reifier

```
PREFIX :    <http://www.example.org/>

:employee38 :familyName "Smith" .
<< :employee38 :jobTitle "Assistant Designer" ~ _:id >> :accordingTo :employee22 .
```

For instance, the syntactic sugar of the above example is equivalent to:

EXAMPLE 25: Reified Triple expanded to Triple Term

```
PREFIX :    <http://www.example.org/>

:employee38 :familyName "Smith" .
_:id rdf:reifies <<( :employee38 :jobTitle "Assistant Designer" )>> .
_:id :accordingTo :employee22 .
```

NOTE

Note the difference in syntax between the syntactic sugar of reifiedTriple (i.e., << [...] >>) and the regular tripleTerm (i.e., <<([...])>>).

After declaring a prefix so that IRIs can be abbreviated, the first triple in this example asserts that employee38 has a familyName of "Smith". Note that this graph does not assert that employee38 has a jobTitle of "Assistant Designer"; it says that employee22 has made that claim using a reifiedTriple. In other words, the triple "employee38 has a jobTitle of 'Assistant Designer'" is not a member of the graph, itself, as "employee38 has a familyName of 'Smith'" is above; rather, it is known as a reifying triple.

A reifiedTriple is syntactic sugar relating a reifier to a tripleTerm using the rdf:reifies predicate.

§ 2.11.1 Annotation Syntax

Turtle also defines an ***annotation syntax*** to both reify and assert a triple, which provides a convenient shortcut. An annotation can be used to simultaneously assert a triple, via an explicit or implicit identifier, and have that triple be the subject or object of further triples. If explicitly identified, the same reifier can then be used as the subject or object of additional triples and/or triple terms. As with a `reifiedTriple`, the annotation syntax allows the definition of one or more reifiers as either IRIs or blank nodes, each preceded by a tilde (~), which precedes the annotation block. If not followed by an annotation block, a reifier is treated like a `reifiedTriple` without annotations. If an annotation block is not immediately preceded by a reifier, an RDF blank node is allocated to serve as the reifier of the triple term.

NOTE

The annotation syntax is a syntactic shortcut in Turtle, and the RDF Abstract Syntax [*RDF11-CONCEPTS*] does not distinguish how the triples were written.

EXAMPLE 26: Annotated Triple

```
VERSION     "1.2"
PREFIX : <http://example.com/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
:a :name "Alice" ~ :t {| :statedBy :bob ; :recorded "2021-07-07"^^xsd:date |} .
```

is the same set of triples as:

EXAMPLE 27: Annotated Triple expanded to Reifiers

```
VERSION     "1.2"
PREFIX : <http://example.com/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
:a :name "Alice" .
<< :a :name "Alice" ~ :t >> :statedBy :bob ;
                            :recorded "2021-07-07"^^xsd:date .
```

and the graph contains three triples. The reifier is identified by `:t`.

Fully expanding to use triple terms, instead of reifiers, results in the following:

EXAMPLE 28: Annotated Triple expanded to Triple Terms

```
VERSION     "1.2"
PREFIX : <http://example.com/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
:a :name "Alice" .
:t rdf:reifies <<( :a :name "Alice" )>> .
:t :statedBy :bob .
:t :recorded "2021-07-07"^^xsd:date .
```

NOTE

An `annotation` can include any number of reifiers.

§ 3. Examples

This section is non-normative.

This example is a Turtle translation of example 7 in *RDF 1.2 XML Syntax* (example1.ttl):

EXAMPLE 29

```
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX ex: <http://example.org/stuff/1.0/>

<https://www.w3.org/TR/rdf12-xml/>
  dc:title "RDF 1.2 XML Syntax" ;
  ex:editor [
    ex:fullName "Gregg Kellogg" ;
    ex:homePage <https://greggkellogg.net/>
  ] .
```

An example of an RDF collection of two literals.

EXAMPLE 30: RDF collection of two literals

```
PREFIX : <http://example.org/stuff/1.0/>
:a :b ( "apple" "banana" ) .
```

which is short for (example2.ttl):

EXAMPLE 31

```
PREFIX : <http://example.org/stuff/1.0/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
:a :b
  [ rdf:first "apple";
    rdf:rest [ rdf:first "banana";
               rdf:rest rdf:nil ]
  ] .
```

An example of two identical triples containing literal objects containing newlines, written in plain and long literal forms. The line breaks in this example are LF. (example3.ttl):

EXAMPLE 32

```
PREFIX : <http://example.org/stuff/1.0/>

:a :b "The first line\nThe second line\n  more" .

:a :b """The first line
The second line
  more""" .
```

As indicated by the grammar, a `collection` can be either a `subject` or an `object`. This subject or object will be the novel blank node for the first object, if the collection has one or more objects, or `rdf:nil` if the collection is empty.

For example,

EXAMPLE 33: Collection used as Subject

```
PREFIX : <http://example.org/stuff/1.0/>
(1 2.0 3E1) :p "w" .
```

is syntactic sugar for (noting that the blank nodes `b0`, `b1`, and `b2` do not occur anywhere else in the RDF graph):

EXAMPLE 34: Collection represented with Blank Nodes

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX : <http://example.org/stuff/1.0/>
_:b0  rdf:first  1 ;
      rdf:rest   _:b1 .
_:b1  rdf:first  2.0 ;
      rdf:rest   _:b2 .
_:b2  rdf:first  3E1 ;
      rdf:rest   rdf:nil .
_:b0  :p         "w" .
```

RDF collections can be nested and can involve other syntactic forms:

EXAMPLE 35: Nested Collections

```
PREFIX : <http://example.org/stuff/1.0/>
(1 [:p :q] ( 2 ) ) :p2 :q2 .
```

is syntactic sugar for:

EXAMPLE 36: Nested Collections represented with Blank Nodes

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX : <http://example.org/stuff/1.0/>
_:b0  :p2  :q2 .
_:b0  rdf:first  1 ;
      rdf:rest   _:b1 .
_:b1  rdf:first  _:b2 .
_:b2  :p         :q .
_:b1  rdf:rest   _:b3 .
_:b3  rdf:first  _:b4 .
_:b4  rdf:first  2 ;
      rdf:rest   rdf:nil .
_:b3  rdf:rest   rdf:nil .
```

§ 4. Turtle compared to SPARQL

This section is non-normative.

The *SPARQL 1.2 Query Language* (SPARQL) [*SPARQL12-QUERY*] uses a Turtle style syntax for its TriplesBlock production. This production differs from the Turtle language in that:

1. SPARQL permits RDF Literals as the subject of Triple Patterns.
2. SPARQL permits variables (`?`*name* or `$`*name*) in any part of the triple of the form.
3. Turtle allows prefix, base, and version declarations anywhere outside of a triple. In SPARQL, they are only allowed in the Prologue (at the start of the SPARQL query).
4. SPARQL uses case insensitive keywords, except for `a`. Turtle's `@prefix` and `@base` declarations are case sensitive, the SPARQL derived `PREFIX` and `BASE` are case insensitive.
5. `true` and `false` are case insensitive in SPARQL and case sensitive in Turtle. `TrUe` is not a valid boolean value in Turtle.

For further information see the Syntax for IRIs and SPARQL Grammar sections of the SPARQL query document [*SPARQL12-QUERY*].

§ 5. Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MAY*, *MUST*, *MUST NOT*, and *SHOULD* in this document are to be interpreted as described in BCP 14 [*RFC2119*] [*RFC8174*] when, and only when, they appear in all capitals, as shown here.

This specification defines conformance criteria for:

- Turtle documents
- Turtle parsers

A conforming ***Turtle document*** is an RDF string that conforms to the grammar and additional constraints defined in 6. Turtle Grammar, starting with the `turtleDoc` production. A Turtle document serializes an RDF graph.

A conforming **Turtle parser** is a system capable of reading Turtle documents on behalf of an application. It makes the serialized RDF graph, as defined in 7. Parsing, available to the application, usually through some form of API.

The IRI that identifies the Turtle language is: `http://www.w3.org/ns/formats/Turtle`

> NOTE
>
> This specification does not define how Turtle parsers handle non-conforming input documents.

§ 5.1 Media Type and Content Encoding

The media type of Turtle is `text/turtle`. The content encoding of Turtle content is always UTF-8 [*RFC3629*]. Charset parameters on the media type are required until such time as the `text/` media type tree permits UTF-8 to be sent without a charset parameter. See D. Internet Media Type, File Extension and Macintosh File Type for the media type registration form.

§ 6. Turtle Grammar

A Turtle document is an RDF string encoded in UTF-8 [*RFC3629*]. Only Unicode scalar values, in the ranges `U+0000` to `U+D7FF` and `U+E000` to `U+10FFFF`, are allowed. This excludes surrogate code points, range `U+D800` to `U+DFFF`.

§ 6.1 White Space

White space (production WS) is used to separate two terminals which would otherwise be (mis-)recognized as one terminal. Rule names below in capitals indicate where white space is significant; these form a possible choice of terminals for constructing a Turtle parser.

White space is significant in the production String.

§ 6.2 Comments

Comments in Turtle start with a # outside an IRIREF, STRING_LITERAL_SINGLE_QUOTE, STRING_LITERAL_QUOTE, STRING_LITERAL_LONG_SINGLE_QUOTE, or STRING_LITERAL_LONG_QUOTE, and continue to the end of line (marked by LF, or CR), or end of file if there is no end of line after the comment marker. Comments are treated as white space.

§ 6.3 IRI References

Relative IRI references are resolved with base IRIs as per *Uniform Resource Identifier (URI): Generic Syntax* [*RFC3986*] using only the basic algorithm in section 5.2. Neither Syntax-Based Normalization nor Scheme-Based Normalization (described in sections 6.2.2 and 6.2.3 of RFC3986) are performed. Characters additionally allowed in IRI references are treated in the same way that unreserved characters are treated in URI references, per section 6.5 of *Internationalized Resource Identifiers (IRIs)* [*RFC3987*].

The @base or BASE directive defines the Base IRI used to resolve relative IRI references per [*RFC3986*] section 5.1.1, "Base URI Embedded in Content". Section 5.1.2, "Base URI from the Encapsulating Entity" defines how the In-Scope Base IRI may come from an encapsulating document, such as a SOAP envelope with an xml:base directive or a MIME multipart document with a Content-Location header. The "Retrieval URI" identified in 5.1.3, Base "URI from the Retrieval URI", is the URL from which a particular Turtle document was retrieved. If none of the above specifies the Base URI, the default Base URI (section 5.1.4, "Default Base URI") is used. Each @base or BASE directive sets a new In-Scope Base URI, relative to the previous one.

§ 6.4 Escape Sequences

There are three forms of escapes used in turtle documents:

- A ***numeric escape sequence*** represents the value of a Unicode code point.

  A numeric escape sequence *MUST NOT* produce a code point value in the range U+D800 to U+DFFF, which is the range for Unicode surrogates.

Escape sequence	Unicode code point
\u hex hex hex hex	A Unicode code point in the ranges U+0000 to U+D7FF and U+E000 to U+D7FF, corresponding to the value encoded by the four hexadecimal digits interpreted from most significant to least significant digit.
\U hex hex hex hex hex hex hex hex	A Unicode code point in the ranges U+0000 to U+D7FF and U+E000 to U+10FFFF, corresponding to the value encoded by the eight hexadecimal digits interpreted from most significant to least significant digit.

where hex is a hexadecimal character

```
HEX ::= [0-9] | [A-F] | [a-f]
```

- A ***string escape sequence*** represents a character traditionally escaped in string literals:

Escape sequence	Unicode code point
`\t`	`U+0009`
`\b`	`U+0008`
`\n`	`U+000A`
`\r`	`U+000D`
`\f`	`U+000C`
`\"`	`U+0022`
`\'`	`U+0027`
`\\`	`U+005C`

- A ***reserved character escape sequence*** consists of a `\` followed by one of these characters `~.-!$&'()*+,;=/?#@%_`, and represents the character to the right of the `\`.

Context where each kind of escape sequence can be used

	numeric escapes	**string escapes**	**reserved character escapes**
IRIs, used as RDF terms or as in `@prefix`, `PREFIX`, `@base`, or `BASE` declarations	yes	no	no
local names	no	no	yes
Strings	yes	yes	no

NOTE

%-encoded sequences are in the character range for IRIs and are explicitly allowed in local names. These appear as a `%` followed by two hex characters and represent that same sequence of three characters. These sequences are *not* decoded during processing. A term written as `<http://a.example/%66oo-bar>` in Turtle designates the IRI `http://a.example/%66oo-bar` and not IRI `http://a.example/foo-bar`. A term written as `ex:%66oo-bar` with a prefix `PREFIX ex: <http://a.example/>` also designates the IRI `http://a.example/%66oo-bar`.

§ 6.5 Grammar

The EBNF used here is defined in XML 1.0 [*EBNF-NOTATION*].

Notes:

1. Keywords in single quotes ('`@base`', '`@prefix`', '`@version`', '`a`', '`true`', and '`false`') are case-sensitive. Keywords in quotation marks ("`BASE`", "`PREFIX`"), and "`VERSION`") are case-insensitive.
2. Escape sequences `UCHAR` and `ECHAR` are case sensitive.
3. When tokenizing the input and choosing grammar rules, the longest match is chosen.
4. The Turtle grammar is LL(1) and LALR(1) when the rules with uppercased names are used as terminals.

5. The entry point into the grammar is `turtleDoc`.
6. In signed numbers, no white space is allowed between the sign and the number.
7. The strings '@prefix', '@base', and '@version' match the pattern for LANG_DIR, though neither `prefix`, `base`, nor `version` are registered language subtags. This specification does not define whether a quoted literal followed by either of these tokens (e.g., `"A"@base`) is in the Turtle language.

```
[1]  turtleDoc                ::= statement*
[2]  statement                ::= directive | (triples '.')
[3]  directive                ::= prefixID | base | version | sparqlPrefix | sparqlBase
                                  | sparqlVersion
[4]  prefixID                 ::= '@prefix' PNAME_NS IRIREF '.'
[5]  base                     ::= '@base' IRIREF '.'
[6]  version                  ::= '@version' VersionSpecifier '.'
[7]  sparqlPrefix             ::= "PREFIX" PNAME_NS IRIREF
[8]  sparqlBase               ::= "BASE" IRIREF
[9]  sparqlVersion            ::= "VERSION" VersionSpecifier
[10] VersionSpecifier         ::= STRING_LITERAL_QUOTE | STRING_LITERAL_SINGLE_QUOTE
[11] triples                  ::= (subject predicateObjectList) | (blankNodePropertyList
                                  predicateObjectList?) | (reifiedTriple
                                  predicateObjectList?)
[12] predicateObjectList      ::= verb objectList (';' (verb objectList)?)*
[13] objectList               ::= object annotation (',' object annotation)*
[14] verb                     ::= predicate | 'a'
[15] subject                  ::= iri | BlankNode | collection
[16] predicate                ::= iri
[17] object                   ::= iri | BlankNode | collection | blankNodePropertyList |
                                  literal | tripleTerm | reifiedTriple
[18] literal                  ::= RDFLiteral | NumericLiteral | BooleanLiteral
[19] blankNodePropertyList    ::= '[' predicateObjectList ']'
[20] collection               ::= '(' object* ')'
[21] NumericLiteral           ::= INTEGER | DECIMAL | DOUBLE
[22] RDFLiteral               ::= String (LANG_DIR | ('^^' iri))?
[23] BooleanLiteral           ::= 'true' | 'false'
[24] String                   ::= STRING_LITERAL_QUOTE | STRING_LITERAL_SINGLE_QUOTE |
                                  STRING_LITERAL_LONG_SINGLE_QUOTE |
                                  STRING_LITERAL_LONG_QUOTE
[25] iri                      ::= IRIREF | PrefixedName
[26] PrefixedName             ::= PNAME_LN | PNAME_NS
[27] BlankNode                ::= BLANK_NODE_LABEL | ANON
[28] reifier                  ::= '~' (iri | BlankNode)?
[29] reifiedTriple            ::= '<<' rtSubject verb rtObject reifier? '>>'
[30] rtSubject                ::= iri | BlankNode | reifiedTriple
[31] rtObject                 ::= iri | BlankNode | literal | tripleTerm | reifiedTriple
[32] tripleTerm               ::= '<<(' ttSubject verb ttObject ')>>'
[33] ttSubject                ::= iri | BlankNode
[34] ttObject                 ::= iri | BlankNode | literal | tripleTerm
[35] annotation               ::= (reifier | annotationBlock)*
[36] annotationBlock          ::= '{|' predicateObjectList '|}'
```

§ Productions for terminals

```
[38] IRIREF                   ::= '<' ([^#x00-#x20<>"{}|^`\] | UCHAR)* '>'
                                  /* #x00=NULL #x01-#x1F=control codes #x20=space */
[39] PNAME_NS                 ::= PN_PREFIX? ':'
[40] PNAME_LN                 ::= PNAME_NS PN_LOCAL
```

```
[41] BLANK_NODE_LABEL                ::= '_:' (PN_CHARS_U | [0-9]) ((PN_CHARS | '.')* PN_CHARS)?
[42] LANG_DIR                        ::= '@' [a-zA-Z]+ ('-' [a-zA-Z0-9]+)* ('--' [a-zA-Z]+)?
[43] INTEGER                         ::= [+-]? [0-9]+
[44] DECIMAL                         ::= [+-]? ([0-9]* '.' [0-9]+)
[45] DOUBLE                          ::= [+-]? (([0-9]+ ('.' [0-9]*)?) | ('.' [0-9]+)) EXPONENT
[46] EXPONENT                        ::= [eE] [+-]? [0-9]+
[47] STRING_LITERAL_QUOTE            ::= '"' ([^#x22#x5C#x0A#x0D] | ECHAR | UCHAR)* '"'
[48] STRING_LITERAL_SINGLE_QUOTE     ::= "'" ([^#x27#x5C#x0A#x0D] | ECHAR | UCHAR)* "'"
[49] STRING_LITERAL_LONG_SINGLE_QUOTE ::= "'''" (("'" | "''")? ([^'\] | ECHAR | UCHAR))* "'''"
[50] STRING_LITERAL_LONG_QUOTE       ::= '"""' (('"' | '""')? ([^"\] | ECHAR | UCHAR))* '"""'
[51] UCHAR                           ::= ('\u' HEX HEX HEX HEX) | ('\U' HEX HEX HEX HEX HEX HEX
                                         HEX HEX)
[52] ECHAR                           ::= '\' [tbnrf\"']
[53] WS                              ::= #x20 | #x09 | #x0D | #x0A
[54] ANON                            ::= '[' WS* ']'
[55] PN_CHARS_BASE                   ::= [A-Z]
                                       | [a-z]
                                       | [#xC0-#xD6]
                                       | [#xD8-#xF6]
                                       | [#xF8-#x02FF]
                                       | [#x0370-#x037D]
                                       | [#x037F-#x1FFF]
                                       | [#x200C-#x200D]
                                       | [#x2070-#x218F]
                                       | [#x2C00-#x2FEF]
                                       | [#x3001-#xD7FF]
                                       | [#xF900-#xFDCF]
                                       | [#xFDF0-#xFFFD]
                                       | [#x00010000-#x000EFFFF]
[56] PN_CHARS_U                      ::= PN_CHARS_BASE | '_'
[57] PN_CHARS                        ::= PN_CHARS_U | '-' | [0-9] | #xB7 | [#x0300-#x036F] |
                                         [#x203F-#x2040]
[58] PN_PREFIX                       ::= PN_CHARS_BASE ((PN_CHARS | '.')* PN_CHARS)?
[59] PN_LOCAL                        ::= (PN_CHARS_U | ':' | [0-9] | PLX) ((PN_CHARS | '.' | ':'
                                         | PLX)* (PN_CHARS | ':' | PLX))?
[60] PLX                             ::= PERCENT | PN_LOCAL_ESC
[61] PERCENT                         ::= '%' HEX HEX
[62] HEX                             ::= [0-9] | [A-F] | [a-f]
[63] PN_LOCAL_ESC                    ::= '\' ('_' | '~' | '.' | '-' | '!' | '$' | '&' | "'" | '('
                                         | ')' | '*' | '+' | ',' | ';' | '=' | '/' | '?' | '#' |
                                         '@' | '%')
```

A text version of this grammar is available here.

§ 6.6 Selected Terminal Literal Strings

This document uses some specific terminal literal strings [*EBNF-NOTATION*]. To clarify the Unicode code points used for these terminal literal strings, the following table describes specific characters and sequences used throughout this document.

Code	Glyph	Description
U+0009	HT	Horizontal tab
U+000A	LF	Line feed
U+000D	CR	Carriage return

U+0022	"	Quotation mark
U+0023	#	Number sign
U+0025	%	Percent sign
U+0027	'	Apostrophe
U+0028	(	Left parenthesis
U+0029	)	Right parenthesis
U+002C	,	Comma
U+002D	-	Hyphen
U+002E	.	Full stop
U+0030	0	Digit zero
U+0039	9	Digit nine
U+003B	:	Colon
U+003B	;	Semicolon
U+003C	<	Less-than sign
U+003E	>	Greater-than sign
U+0040	@	At sign
U+0045	E	Latin capital letter E
U+005B	[	Left square bracket
U+005C	\	Backslash
U+005D	[	Right square bracket
U+005F	_	Underscore
U+0061	a	Latin small letter A
U+0065	e	Latin small letter E
U+007C	|	vertical line
U+007E	~	Tilde
U+00B7	·	Middle dot
U+203F	‿	Undertie
U+2040	⁀	Character tie

Other short terminal literal strings are composed of specific sequences of Unicode characters:

space
: U+0020

"""
: three concatenated quotation mark characters, each having the code point U+0022

'''
: three concatenated apostrophes characters, each having the code point U+0027

<<
: two concatenated less-than sign characters, each having the code point U+003C

>>
: two concatenated greater-than sign characters, each having the code point U+003E

<<(
: two concatenated less-than sign characters, each having the code point U+003C, followed by a left parenthesis character, having the code point U+0028

)>>
: a left parenthesis character, having the code point U+0029 followed by two concatenated greater-than sign characters, each having the code point U+003E

^^

two concatenated circumflex accent characters, each having the code point U+005E

`{|`

{ (left curly bracket, code point U+007B) followed by | (vertical line, code point U+007C)

`|}`

| (vertical line, code point U+007C) followed by } (right curly bracket, code point U+007D)

`_:`

_ followed by :

`--`

two concatenated - characters

§ 7. Parsing

The RDF 1.2 Concepts and Abstract Syntax specification [*RDF12-CONCEPTS*] defines four types of RDF terms: IRIs, literals, blank nodes, and triple terms. Literals are composed of a lexical form and an optional language tag [*BCP47*] – possibly including an initial text direction – or datatype IRI. An extra type, `prefix`, is used during parsing to map string identifiers to namespace IRIs. This section maps a string conforming to the grammar in 6.5 Grammar to a set of triples by mapping strings matching productions and lexical tokens to RDF terms or their components (e.g., language tags, lexical forms of literals). Grammar productions change the parser state and emit triples.

§ 7.1 Parser State

Parsing Turtle requires a state of nine items:

- IRI *baseURI* — When the `base` production is reached, the second rule argument, `IRIREF`, is the base URI used for relative IRI resolution.
- Map[prefix -> IRI] *namespaces* — The second and third rule arguments (`PNAME_NS` and `IRIREF`) in the `prefixID` production assign a namespace name (`IRIREF`) for the prefix (`PNAME_NS`). Outside of a `prefixID` or `sparqlPrefix` production, any `PNAME_NS` is substituted with the namespace from the current state of the namespaces map. Note that the prefix may be an empty string, per the `PNAME_NS` production: `PN_PREFIX? ':'`.
- Map[string -> blank node] *bnodeLabels* — A mapping from a string to a blank node.
- RDF Term *curSubject* — The *curSubject* is bound to the `subject`, `rtSubject`, `ttSubject`, `blankNodePropertyList`, `collection`,and `annotationBlock` productions.
- RDF Term *curPredicate* — The *curPredicate* is bound to the `verb` production. If token matched was `a`, *curPredicate* is bound to the IRI `http://www.w3.org/1999/02/22-rdf-syntax-ns#type`.
- RDF Term *curObject* — The *curObject* is bound to the `object`, `rtObject`, and `ttObject` productions.
- RDF Term *curReifier* — The *curReifier* is bound to the `reifier` and `annotationBlock` productions.
- Triple term *curTripleTerm* — The *curTripleTerm* is set in the Annotations constructor.
- `xsd:string` *curVersion* – The RDF version used for parsing the document into Triples. If specified as part of a Media Type, the default value for *curVersion* is taken from the `version` parameter. Acceptable values for *curVersion* are defined in 2.1 Version Labels in [*RDF12-CONCEPTS*]. The version announcement is only a hint; this specification does not mandate any parser behavior based on *curVersion*, but a parser *MAY* signal an error or a warning when it encounters a feature that does not match the value of *curVersion*, or an unacceptable value for *curVersion*.

Term Constructors can create a stack of these values indicated by using language such as "records the *curSubject* and *curPredicate*."

§ 7.2 RDF Term Constructors

This table maps productions and lexical tokens to RDF terms or components of RDF terms listed in 7. Parsing:

production	type	procedure
IRIREF	IRI	The characters between < and > are taken, with the numeric escape sequences unescaped, to form the IRI. Relative IRI reference resolution is performed per Section 6.3. The resulting IRI *MUST* comply with the syntactic restrictions of generic IRI syntax, and *SHOULD* conform to section 3.3 of [*RFC3986*] and comply with any narrower restrictions imposed by the corresponding IRI scheme specification.
PNAME_NS	prefix	When used in a `prefixID` or `sparqlPrefix` production, the `prefix` is the potentially empty RDF string matching the first argument of the rule is a key into the namespaces map into which the expanded second argument is stored for future lookup.
	IRI	When used in a `PrefixedName` production; the namespaces map *MUST* have a corresponding `namespace`, which forms the RDF string of the IRI. The resulting IRI *MUST* comply with the syntactic restrictions of generic IRI syntax, and *SHOULD* comply with any narrower restrictions imposed by the corresponding IRI scheme specification.
PNAME_LN	IRI	A potentially empty prefix is identified by the first sequence, `PNAME_NS`. The namespaces map *MUST* have a corresponding `namespace`. The RDF string of the IRI is formed by unescaping the reserved characters in the second argument, `PN_LOCAL`, and concatenating this onto the `namespace`. The resulting IRI *MUST* comply with the syntactic restrictions of generic IRI syntax, and *SHOULD* comply with any narrower restrictions imposed by the corresponding IRI scheme specification.
VersionSpecifier	literal.	The *curVersion* is taken from a literal using the matched RDF string lexical form and `xsd:string` datatype.
STRING_LITERAL_SINGLE_QUOTE	lexical form	The characters between the outermost '''s are taken, after numeric and string escape sequences are replaced with the characters that they represent, to form the RDF string of a lexical form.
STRING_LITERAL_QUOTE	lexical form	The characters between the outermost """s are taken, with numeric and string escape

		sequences unescaped, to form the RDF string of a lexical form.
STRING_LITERAL_LONG_SINGLE_QUOTE	lexical form	The characters between the outermost `'''`s are taken, with numeric and string escape sequences unescaped, to form the RDF string of a lexical form.
STRING_LITERAL_LONG_QUOTE	lexical form	The characters between the outermost `"""`s are taken, after numeric and string escape sequences are are replaced with the characters that they represent, to form the RDF string of a lexical form.
LANG_DIR	language tag	The characters following the `@` form the language tag and optionally the initial text direction, if the matched characters include `--`. The language tag *MUST* be well-formed according to section 2.2.9 of [*BCP47*]. If present, the initial text direction *MUST* be either `ltr` or `rtl`.
RDFLiteral	literal	The literal has a lexical form of the first rule argument, `String`. If the `'^^' iri` rule is matched, the datatype IRI is derived from the `iri`, and the literal has no language tag. If the `LANG_DIR` rule is matched, the language tag and initial text direction are taken from `LANG_DIR`. If there is no initial text direction, the datatype is `rdf:langString`. If there is a initial text direction, the datatype is `rdf:dirLangString`. If neither matched, the datatype is `xsd:string`, and the literal has no language tag.
INTEGER	literal	The literal has a lexical form of the input string, and a datatype of `xsd:integer`.
DECIMAL	literal	The literal has a lexical form of the input string, and a datatype of `xsd:decimal`.
DOUBLE	literal	The literal has a lexical form of the input string, and a datatype of `xsd:double`.
BooleanLiteral	literal	The literal has a lexical form of the `true` or `false`, depending on which matched the input, and a datatype of `xsd:boolean`.
BLANK_NODE_LABEL	blank node	The string matching the second argument, `PN_LOCAL`, is a key in bnodeLabels. If there is no corresponding blank node in the map, one is allocated.
ANON	blank node	A blank node is generated.
blankNodePropertyList	blank node	A blank node is generated. Note the rules for `blankNodePropertyList` in the next

		section.
collection	blank node	For non-empty lists, a blank node is generated. Note the rules for `collection` in the next section.
	IRI	For empty lists, the resulting IRI is `rdf:nil`. Note the rules for `collection` in the next section.
reifier	IRI | blank node	The *curReifier* is taken from term, which is taken from the matched `iri` production or `BlankNode` production, if any. If no such production is matched, term is taken from a fresh RDF blank node.
tripleTerm	triple term	The triple term is composed of the terms constructed from the `ttSubject`, `predicate`, and `ttObject` productions.
reifiedTriple	IRI | blank node	The term is taken from the matched `reifier`, if any, or from a fresh RDF blank node.
annotationBlock	IRI | blank node	The term is taken from a previously matched reifier, if any, or from a fresh RDF blank node.

NOTE

As processors which detect errors on input can result in graphs which contain fewer triples than are described in the input (including no triples whatsoever), consumers should consider information of any errors signaled when using the output triples, which may be incomplete and/or include ill-typed or ill-formed terms.

§ 7.3 RDF Triples Constructors

A Turtle document defines an RDF graph composed of set of RDF triples. The `subject` production sets the *curSubject*. The `verb` production sets the *curPredicate*. The `object` and `ttObject` productions set the *curObject*. Each `object` *N* in the document produces an RDF triple: *curSubject curPredicate N* .

§ 7.3.1 Reifiers

Beginning the `reifier` production, the *curReifier* is taken from the `reifier` term constructor. Then yield the RDF triple *curReifier* `rdf:reifies` *curTripleTerm*.

§ 7.3.2 Reified Triples

Beginning the `reifiedTriple` production records the *curTripleTerm*. A new `tripleTerm` instance *curTripleTerm* is created using the `rtSubject`, `verb`, and `rtObject` productions. Finishing the `reifiedTriple` production, if the *curReifier* is not set, it is assigned a fresh RDF blank node; it next yields the RDF triple *curReifier* `rdf:reifies` *curTripleTerm*, and then restores the recorded value of the *curTripleTerm*. The node produced by matching `reifiedTriple` is the the *curReifier*.

§ 7.3.3 Annotations

Beginning the `annotation` production records the *curSubject* and *curPredicate*. A new `tripleTerm` instance *curTripleTerm* is created using the *curSubject curPredicate curObject*, and the value of the *curReifier* is cleared. Finishing the `annotation` production restores the recorded values of the *curSubject* and *curPredicate*.

§ 7.3.4 Annotation Blocks

Beginning the `annotationBlock` production records the *curTripleTerm*. If the *curReifier* is not set, then it is assigned a fresh RDF blank node and the production yields the RDF triple *curReifier* `rdf:reifies` *curTripleTerm*. The *curSubject* is taken from the *curReifier* Finishing the `annotationBlock` production clears the value of the *curReifier* and restores the *curTripleTerm*.

> NOTE
>
> If the *curReifier* was already set, the reifying triple *curReifier* `rdf:reifies` *curTripleTerm* was emitted in 7.3.1 Reifiers.

§ 7.3.5 Property Lists

Beginning the `blankNodePropertyList` production records the *curSubject* and *curPredicate*, and sets the *curSubject* to a novel blank node *B*. Finishing the `blankNodePropertyList` production restores the *curSubject* and *curPredicate*. The node produced by matching `blankNodePropertyList` is the blank node *B*.

§ 7.3.6 Collections

Beginning the `collection` production records the *curSubject* and *curPredicate*. Each `object` in the `collection` production has *curSubject* set to a novel blank node *B* and *curPredicate* set to `rdf:first`. For each object $object_n$ after the first produces a triple:$object_{n-1}$ `rdf:rest` $object_n$. Finishing the `collection` production creates an additional triple `curSubject rdf:rest rdf:nil` . and restores the *curSubject* and *curPredicate* The node produced by matching `collection` is the first blank node *B* for non-empty lists and `rdf:nil` for empty lists.

§ 7.4 Parsing Example

This section is non-normative.

The following informative example shows the semantic actions performed when parsing this Turtle document with an LALR(1) parser:

> EXAMPLE 37: Example of parsing Turtle

```
PREFIX ericFoaf: <http://www.w3.org/People/Eric/ericP-foaf.rdf#>
PREFIX : <http://xmlns.com/foaf/0.1/>
ericFoaf:ericP :givenName "Eric" ;
  :knows <http://norman.walsh.name/knows/who/dan-brickley> ,
          [ :mbox <mailto:timbl@w3.org> ] ,
          <http://getopenid.com/amyvdh> .
```

- Map the prefix `ericFoaf` to the IRI `http://www.w3.org/People/Eric/ericP-foaf.rdf#`.

- Map the empty prefix to the IRI `http://xmlns.com/foaf/0.1/`.
- Assign *curSubject* the IRI `http://www.w3.org/People/Eric/ericP-foaf.rdf#ericP`.
- Assign *curPredicate* the IRI `http://xmlns.com/foaf/0.1/givenName`.
- Emit an RDF triple: `<...rdf#ericP> <.../givenName> "Eric" .`
- Assign *curPredicate* the IRI `http://xmlns.com/foaf/0.1/knows`.
- Emit an RDF triple: `<...rdf#ericP> <.../knows> <...who/dan-brickley> .`
- Emit an RDF triple: `<...rdf#ericP> <.../knows> _:1 .`
- Save *curSubject* and reassign to the blank node `_:1`.
- Save *curPredicate*.
- Assign *curPredicate* the IRI `http://xmlns.com/foaf/0.1/mbox`.
- Emit an RDF triple: `_:1 <.../mbox> <mailto:timbl@w3.org> .`
- Restore *curSubject* and *curPredicate* to their saved values (`<...rdf#ericP>`, `<.../knows>`).
- Emit an RDF triple: `<...rdf#ericP> <.../knows> <http://getopenid.com/amyvdh> .`

§ A. Embedding Turtle in HTML documents

This section is non-normative.

HTML [*HTML5*] `script` tags can be used to embed data blocks in documents. Turtle can be easily embedded in HTML this way.

EXAMPLE 38: Embedding Turtle in HTML

```
<script type="text/turtle">
  PREFIX dc: <http://purl.org/dc/terms/>
  PREFIX frbr: <http://purl.org/vocab/frbr/core#>

  <http://books.example.com/works/45U8QJGZSQKDH8N> a frbr:Work ;
    dc:creator "Wil Wheaton"@en ;
    dc:title "Just a Geek"@en ;
    frbr:realization <http://books.example.com/products/9780596007683.BOOK>,
      <http://books.example.com/products/9780596802189.EBOOK> .

  <http://books.example.com/products/9780596007683.BOOK> a frbr:Expression ;
    dc:type <http://books.example.com/product-types/BOOK> .

  <http://books.example.com/products/9780596802189.EBOOK> a frbr:Expression ;
    dc:type <http://books.example.com/product-types/EBOOK> .
</script>
```

Turtle content should be placed in a `script` tag with the `type` attribute set to `text/turtle`. `<` and `>` symbols do not need to be escaped inside of script tags. The character encoding of the embedded Turtle will match the HTML documents encoding.

§ A.1 XHTML

This section is non-normative.

Like JavaScript, Turtle authored for HTML (`text/html`) can break when used in XHTML (`application/xhtml+xml`). The solution is the same one used for JavaScript.

EXAMPLE 39: Embedding Turtle in XHTML

```
<script type="text/turtle">
  # <![CDATA[
  PREFIX frbr: <http://purl.org/vocab/frbr/core#>

  <http://books.example.com/works/45U8QJGZSQKDH8N> a frbr:Work .
  # ]]>
</script>
```

When embedded in XHTML Turtle data blocks must be enclosed in CDATA sections. Those CDATA markers must be in Turtle comments. If the character sequence `]]>` occurs in the document it must be escaped using strings escapes (`\u005d\u0054\u003e`). This will also make Turtle safe in polyglot documents served as both `text/html` and `application/xhtml+xml`. Failing to use CDATA sections or escape `]]>` may result in a non well-formed XML document.

§ A.2 Parsing Turtle in HTML

This section is non-normative.

There are no syntactic or grammar differences between parsing Turtle that has been embedded and normal Turtle documents. A Turtle document parsed from an HTML DOM will be a stream of character data rather than a stream of UTF-8 [*RFC3629*] code points. No decoding is necessary if the HTML document has already been parsed into DOM. Each `script` data block is considered to be its own Turtle document. `PREFIX` and `BASE` declarations in a Turtle data block are scoped to that data block and do not effect other data blocks. The HTML `lang` attribute or XHTML `xml:lang` attribute have no effect on the parsing of the data blocks. The base URI of the encapsulating HTML document provides a "Base URI Embedded in Content" per RFC3986 section 5.1.1.

§ B. Privacy Considerations

This section is non-normative.

The Turtle format is used to express arbitrary application data, which may include the expression of personally identifiable information (PII) or other information which could be considered sensitive. Authors publishing such information are advised to carefully consider the needs and use of publishing such information, as well as the applicable regulations for the regions where the data is expected to be consumed and potentially revealed (e.g., GDPR, CCPA, others), particularly whether authorization measures are needed for access to the data.

§ C. Security Considerations

This section is non-normative.

The `STRING_LITERAL_SINGLE_QUOTE`, `STRING_LITERAL_QUOTE`, `STRING_LITERAL_LONG_SINGLE_QUOTE`, and `STRING_LITERAL_LONG_QUOTE`, productions allow the use of unescaped control characters. Although this specification does not directly expose this content to an end user, it might be presented through a user agent, which may cause the presented text to be obfuscated due to presentation of such characters.

Turtle is a general-purpose assertion language; applications may evaluate given data to infer more assertions or to dereference IRIs, invoking the security considerations of the scheme for that IRI. Note in particular, the privacy issues in [*RFC3023*] section 10 for HTTP IRIs. Data obtained from an inaccurate or malicious data source may lead to inaccurate or misleading conclusions, as well as the dereferencing of unintended IRIs. Care must be taken to align the trust in consulted resources with the sensitivity of the intended use of the data; inferences of potential medical treatments would likely require different trust than inferences for trip planning.

The Turtle language is used to express arbitrary application data; security considerations will vary by domain of use. Security tools and protocols applicable to text (for example, PGP encryption, checksum validation, password-protected compression) may also be used on Turtle documents. Security/privacy protocols must be imposed which reflect the sensitivity of the embedded information.

Turtle can express data which is presented to the user, such as RDF Schema labels. Applications rendering strings retrieved from untrusted Turtle documents, or using unescaped characters, *SHOULD* use warnings and other appropriate means to limit the possibility that malignant strings might be used to mislead the reader. The security considerations in the media type registration for XML ([*RFC3023*] section 10) provide additional guidance around the expression of arbitrary data and markup.

Turtle uses IRIs as term identifiers. Applications interpreting data expressed in Turtle *SHOULD* address the security issues of *Internationalized Resource Identifiers (IRIs)* [*RFC3987*] Section 8, as well as *Uniform Resource Identifier (URI): Generic Syntax* [*RFC3986*] Section 7.

Multiple IRIs may have the same appearance. Characters in different scripts may look similar (for instance, a Cyrillic "o" (code point `U+043E`) may appear similar to a Latin "o" (code point `U+006F`)). A character followed by combining characters may have the same visual representation as another character (for example, LATIN SMALL LETTER "E" (code point `U+0065`) followed by COMBINING ACUTE ACCENT (code point `U+0301`) has the same visual representation as LATIN SMALL LETTER "E" WITH ACUTE (`U+00E9`)). Any person or application that is writing or interpreting data in Turtle must take care to use the IRI that matches the intended semantics, and avoid IRIs that may look similar. Further information about matching visually similar characters can be found in *Unicode Security Considerations* [*UNICODE-SECURITY*] and *Internationalized Resource Identifiers (IRIs)* [*RFC3987*] Section 8.

§ D. Internet Media Type, File Extension and Macintosh File Type

The Internet Media Type (formerly known as MIME Type) for Turtle is "text/turtle".

The information that follows has been submitted to the Internet Engineering Steering Group (IESG) for review, approval, and registration with IANA.

Type name:
text

Subtype name:
turtle

Required parameters:
None

Optional parameters:
`charset` — this parameter is required when transferring non-ASCII data. If present, the value of `charset` is always `UTF-8`.

`version` — this parameter is optional but *SHOULD* be present when using RDF 1.2-specific features. If present, acceptable values of `version` are defined in 2.1 Version Labels in [*RDF12-CONCEPTS*].

Encoding considerations:
The syntax of Turtle is expressed over code points in Unicode [*UNICODE*]. The encoding is always UTF-8 [*UTF-8*].

Unicode code points may also be expressed using `\uXXXX` (`U+0000` to `U+FFFF`) or `\UXXXXXXXX` syntax (for code points up to `U+10FFFF`) where X is a hexadecimal digit `[0-9A-Fa-f]`

Security considerations:

See C. Security Considerations.

Interoperability considerations:
There are no known interoperability issues.

Published specification:
This specification.

Applications which use this media type:
Turtle is used widely for representing RDF data. There are implementations available in most common programming languages.

Additional information:

Magic number(s):
Turtle documents may have the strings '@prefix' or '@base' (case sensitive) or the strings 'PREFIX' or 'BASE' (case insensitive) near the beginning of the document.

File extension(s):
.ttl

Macintosh file type code(s):
TEXT

Base URI:
The Turtle '@base <IRIref>' or 'BASE <IRIref>' term can change the current base URI for relative IRI references in the query language that are used sequentially later in the document.

Person & email address to contact for further information:
RDF & SPARQL Working Group <public-rdf-star-wg@w3.org>

Intended usage:
Common

Restrictions on usage:
None

Author(s):
The Turtle specification is the product of the RDF & SPARQL WG. The W3C reserves change control over this specifications.

E. Acknowledgments

E.1 Acknowledgments for RDF 1.1

This section is non-normative.

This work was described in the paper New Syntaxes for RDF which discusses other RDF syntaxes and the background to the Turtle (Submitted to WWW2004, referred to as *N-Triples Plus* there).

This work was started during the Semantic Web Advanced Development Europe (SWAD-Europe) project funded by the EU IST-7 programme IST-2001-34732 (2002-2004) and further development supported by the Institute for Learning and Research Technology at the University of Bristol, UK (2002-Sep 2005).

Valuable contributions to this version were made by Gregg Kellogg, Andy Seaborne, Sandro Hawke and the members of the RDF Working Group.

The document was improved through the review process by the wider community.

E.2 Acknowledgments for RDF 1.2

This section is non-normative.

In addition to the editors, the following people have contributed to this specification: Andrew Louis, Andy Seaborne, Denis Ah-Kang, John Walker, Mirek Kratochvil, Peter F. Patel-Schneider, Pierre-Antoine Champin, Ted Thibodeau Jr, and William Van Woensel

Members of the RDF & SPARQL Working Group Group included Vladimir Alexiev, James Anderson, Amin Anjomshoaa, Julián Arenas-Guerrero, Dörthe Arndt, Bilal Ben Mahria, Erich Bremer, Dan Brickley, Kurt Cagle, Sarven Capadisli, Rémi Ceres, Pierre-Antoine Champin, David Chaves-Fraga, Souripriya Das, Daniil Dobriy, Enrico Franconi, Jeffrey Phillips Freeman, Fabien Gandon, Benjamin Goering, Damien Graux, Adrian Gschwend, Olaf Hartig, Timothée Haudebourg, Ian Horrocks, Gregg Kellogg, Mark Kim, Jose Emilio Labra Gayo, Ora Lassila, Richard Lea, Niklas Lindström, Pasquale Lisena, Thomas Lörtsch, Matthew Nguyen, Peter Patel-Schneider, Thomas Pellissier Tanon, Dave Raggett, Jean-Yves ROSSI, Felix Sasaki, Andy Seaborne, Alan Snyder, Stuart Sutton, Ruben Taelman, Ted Thibodeau Jr, Dominik Tomaszuk, Raphaël Troncy, William Van Woensel, Gregory Williams, Jesse Wright, Achille Zappa, and Antoine Zimmermann.

> EDITOR'S NOTE
>
> Recognize members of the Task Force? Not an easy to find list of contributors.

§ F. Changes between RDF 1.1 and RDF 1.2

This section is non-normative.

- Update informative restrictions on included characters for `STRING_LITERAL_QUOTE` and `STRING_LITERAL_SINGLE_QUOTE`.
- Separated C. Security Considerations from D. Internet Media Type, File Extension and Macintosh File Type and updated language.
- Added 2.10 Triple Terms, 2.11 Reifying Triples, and 2.11.1 Annotation Syntax for representing triple terms in Turtle.
- Changes the `LANGTAG` terminal production to `LANG_DIR` to include an optional initial text direction.
- Clarify that Unicode surrogates are not legal in Turtle documents, nor as the value of a Unicode escape sequence.
- Added 2.4 Version Announcement and parser state and productions to announce the RDF version associated with the input document. Note that this also includes a provision that `@version` does not match `LANG_DIR`; the previous version of the grammar would allow such a match, but would result in a literal with an ill-formed language tag according to [*BCP47*], and form an invalid literal.

§ G. Index

§ G.1 Terms defined by this specification

annotation syntax §2.11.1

numeric escape sequence §6.4

prefixed name §2.5

reserved character escape sequence §6.4

string escape sequence §6.4

Turtle document §5.

§ G.2 Terms defined by reference

[*BCP47*] defines the following:

- section 2.2.9

[*I18N-GLOSSARY*] defines the following:

- surrogate code points
- Unicode code point
- Unicode scalar values

[*LANG-SUBTAG-REGISTRY*] defines the following:

- registered language subtags

[*RDF-MT*] defines the following:

- RDF Semantics

[*RDF12-CONCEPTS*] defines the following:

- 2.1 Version Labels
- blank node identifier
- blank nodes
- datatype IRI
- directional language-tagged strings
- ill-typed
- initial text direction
- IRIs
- language tag
- literals
- object type
- predicate
- RDF graph
- RDF lexical form
- RDF string
- RDF term
- reifier
- reifying triple
- Relative IRI references
- subject
- triple
- triple terms

[*RDF12-SEMANTICS*] defines the following:

- Collection
- Reification vocabulary

[*RDF12-XML*] defines the following:

- example 7

[*RFC3986*] defines the following:

- 5.1.3, Base "URI from the Retrieval URI"
- resolved
- section 3.3
- section 5.1.1, "Base URI Embedded in Content"
- Section 5.1.2, "Base URI from the Encapsulating Entity"
- section 5.1.4, "Default Base URI"

[*RFC3987*] defines the following:

- syntactic restrictions

[*SPARQL12-QUERY*] defines the following:

- Prologue
- SPARQL Grammar
- Syntax for IRIs
- Triple P

Triple Patterns

TriplesBlock

[*XML-NAMES*] defines the following:

QNames

[*XMLSCHEMA11-2*] defines the following:

xsd:boolean

§ H. References

§ H.1 Normative references

[BCP47]
Tags for Identifying Languages. A. Phillips, Ed.; M. Davis, Ed. IETF. September 2009. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc5646

[EBNF-NOTATION]
EBNF Notation. Tim Bray; Jean Paoli; Michael Sperberg-McQueen; Eve Maler; François Yergeau et al. W3C. W3C Recommendation. URL: https://www.w3.org/TR/xml/#sec-notation

[I18N-GLOSSARY]
Internationalization Glossary. Richard Ishida; Addison Phillips. W3C. 17 October 2024. W3C Working Group Note. URL: https://www.w3.org/TR/i18n-glossary/

[RDF12-CONCEPTS]
RDF 1.2 Concepts and Abstract Data Model. Gregg Kellogg; Olaf Hartig; Pierre-Antoine Champin; Andy Seaborne. W3C. 28 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-concepts/

[RFC2119]
Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc2119

[RFC3629]
UTF-8, a transformation format of ISO 10646. F. Yergeau. IETF. November 2003. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3629

[RFC3986]
Uniform Resource Identifier (URI): Generic Syntax. T. Berners-Lee; R. Fielding; L. Masinter. IETF. January 2005. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3986

[RFC3987]
Internationalized Resource Identifiers (IRIs). M. Duerst; M. Suignard. IETF. January 2005. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc3987

[RFC8174]
Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc8174

[UNICODE]
The Unicode Standard. Unicode Consortium. URL: https://www.unicode.org/versions/latest/

§ H.2 Informative references

[HTML5]
HTML5. Ian Hickson; Robin Berjon; Steve Faulkner; Travis Leithead; Erika Doyle Navara; Theresa O'Connor; Silvia Pfeiffer. W3C. 27 March 2018. W3C Recommendation. URL: https://www.w3.org/TR/html5/

[LANG-SUBTAG-REGISTRY]

Language Subtag Registry. IANA. URL: http://www.iana.org/assignments/language-subtag-registry/language-subtag-registry

[RDF-MT]
RDF Semantics. Patrick Hayes. W3C. 10 February 2004. W3C Recommendation. URL: https://www.w3.org/TR/rdf-mt/

[RDF11-CONCEPTS]
RDF 1.1 Concepts and Abstract Syntax. Richard Cyganiak; David Wood; Markus Lanthaler. W3C. 25 February 2014. W3C Recommendation. URL: https://www.w3.org/TR/rdf11-concepts/

[RDF12-INTEROP]
RDF 1.2 Interoperability. Pierre-Antoine Champin. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-interop/spec/

[RDF12-N-QUADS]
RDF 1.2 N-Quads. Gregg Kellogg; Dominik Tomaszuk. W3C. 22 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-quads/

[RDF12-N-TRIPLES]
RDF 1.2 N-Triples. Gregg Kellogg; Dominik Tomaszuk. W3C. 20 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-triples/

[RDF12-NEW]
What's New in RDF 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-new/spec/

[RDF12-PRIMER]
RDF 1.2 Primer. Niklas Lindström; Pierre-Antoine Champin. W3C. 3 April 2025. DNOTE. URL: https://www.w3.org/TR/rdf12-primer/

[RDF12-SCHEMA]
RDF 1.2 Schema. Dominik Tomaszuk. W3C. 26 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-schema/

[RDF12-SEMANTICS]
RDF 1.2 Semantics. Peter Patel-Schneider; Dörthe Arndt; Enrico Franconi. W3C. 28 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-semantics/

[RDF12-TRIG]
RDF 1.2 TriG. Gregg Kellogg; Dominik Tomaszuk. W3C. 13 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-trig/

[RDF12-XML]
RDF 1.2 XML Syntax. Gregg Kellogg. W3C. 19 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-xml/

[RFC3023]
XML Media Types. M. Murata; S. St. Laurent; D. Kohn. IETF. January 2001. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc3023

[SPARQL12-CONCEPTS]
SPARQL 1.2 Concepts. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-concepts/spec/

[SPARQL12-ENTAILMENT]
SPARQL 1.2 Entailment Regimes. Peter Patel-Schneider. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-entailment/

[SPARQL12-FEDERATED-QUERY]
SPARQL 1.2 Federated Query. Ruben Taelman; Gregory Williams. W3C. 26 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-federated-query/

[SPARQL12-GRAPH-STORE-PROTOCOL]
SPARQL 1.2 Graph Store Protocol. Andy Seaborne; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-graph-store-protocol/

[SPARQL12-NEW]
What's New in SPARQL 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-new/spec/

[SPARQL12-PROTOCOL]

SPARQL 1.2 Protocol. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-protocol/

[SPARQL12-QUERY]

SPARQL 1.2 Query Language. Olaf Hartig; Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 23 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-query/

[SPARQL12-RESULTS-CSV-TSV]

SPARQL 1.2 Query Results CSV and TSV Formats. Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-csv-tsv/

[SPARQL12-RESULTS-JSON]

SPARQL 1.2 Query Results JSON Format. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-json/

[SPARQL12-RESULTS-XML]

SPARQL 1.2 Query Results XML Format. Ruben Taelman; Dominik Tomaszuk; Thomas Pellissier Tanon. W3C. 27 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-xml/

[SPARQL12-SERVICE-DESCRIPTION]

SPARQL 1.2 Service Description. Ruben Taelman; Gregory Williams. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-service-description/

[SPARQL12-UPDATE]

SPARQL 1.2 Update. Ruben Taelman; Andy Seaborne; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-update/

[TURTLE]

RDF 1.1 Turtle. Eric Prud'hommeaux; Gavin Carothers. W3C. 25 February 2014. W3C Recommendation. URL: https://www.w3.org/TR/turtle/

[UNICODE-SECURITY]

Unicode Security Considerations. Mark Davis; Michel Suignard. Unicode Consortium. 19 September 2014. Unicode Technical Report #36. URL: https://www.unicode.org/reports/tr36/tr36-15.html

[XML-NAMES]

Namespaces in XML 1.0 (Third Edition). Tim Bray; Dave Hollander; Andrew Layman; Richard Tobin; Henry Thompson et al. W3C. 8 December 2009. W3C Recommendation. URL: https://www.w3.org/TR/xml-names/

[XMLSCHEMA11-2]

W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes. David Peterson; Sandy Gao; Ashok Malhotra; Michael Sperberg-McQueen; Henry Thompson; Paul V. Biron et al. W3C. 5 April 2012. W3C Recommendation. URL: https://www.w3.org/TR/xmlschema11-2/

↥